UNIVILLE – UNIVERSIDADE DA REGIÃO DE JOINVILLE

Atividade – Resolução de Problemas

Problema do Caixeiro Viajante: resolução por Força Bruta

Integrante: Gustavo Gorges

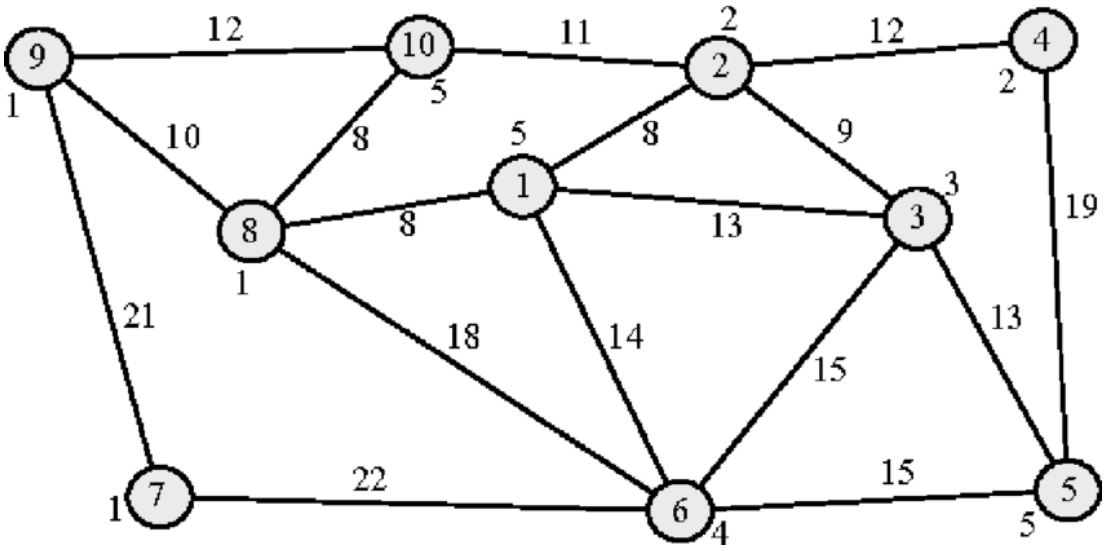
Outubro de 2026

1. Descrição do problema

O Problema do Caixeiro Viajante (PCV, em inglês *Traveling Salesman Problem*) consiste em, dado um conjunto de cidades e o custo de deslocamento entre elas, encontrar a rota de menor custo total que parte de uma cidade, visita cada uma das demais exatamente uma vez e retorna à cidade de origem. Em termos de grafos, as cidades são vértices, as estradas são arestas com peso, e procura-se o **ciclo hamiltoniano de menor peso**. O PCV é NP-difícil: não se conhece algoritmo exato que o resolva em tempo polinomial, por isso instâncias pequenas costumam ser resolvidas testando todas as possibilidades.

A instância proposta na atividade (Figura 1) tem **10 cidades**, numeradas de 1 a 10, e **17 estradas**, cada uma com o seu custo. As estradas podem ser percorridas nos dois sentidos com o mesmo custo. O grafo **não é completo**: nem todo par de cidades tem estrada direta (não há, por exemplo, estrada entre 1 e 5). Por isso, uma ordem de visita só é uma rota válida se todos os trechos consecutivos existirem no mapa. Os números pequenos ao lado de cada vértice não representam custo de deslocamento e não foram usados, pois o PCV considera apenas o custo das estradas.

Figura 1 – Grafo de cidades e custos das estradas



Fonte: enunciado da atividade.

Tabela 1 – Entrada do Exemplo 2: lista de adjacência do grafo da Figura 1

Cidade	Cidades vizinhas (custo da estrada)	Grau
1	2 (8), 3 (13), 6 (14), 8 (8)	4
2	1 (8), 3 (9), 4 (12), 10 (11)	4
3	1 (13), 2 (9), 5 (13), 6 (15)	4
4	2 (12), 5 (19)	2
5	3 (13), 4 (19), 6 (15)	3
6	1 (14), 3 (15), 5 (15), 7 (22), 8 (18)	5
7	6 (22), 9 (21)	2
8	1 (8), 6 (18), 9 (10), 10 (8)	4
9	7 (21), 8 (10), 10 (12)	3
10	2 (11), 8 (8), 9 (12)	3

Fonte: elaborado pelo autor a partir da Figura 1.

2. Solução proposta

2.1 Técnica escolhida: Força Bruta (busca exaustiva)

A Força Bruta resolve o problema da forma mais direta: gera todas as soluções candidatas, avalia cada uma e guarda a melhor. No PCV, cada candidata é uma ordem de visita das cidades. Como a rota é um ciclo, a cidade inicial pode ser fixada sem perder nenhuma solução: o mesmo ciclo começando em outra cidade tem exatamente o mesmo custo. Assim, para n cidades, basta testar as $(n - 1)!$ permutações das demais. Para as 10 cidades da atividade, são 9! = **362.880 permutações**.

A Força Bruta foi escolhida porque o enunciado pede que todas as permutações sejam testadas e porque ela **garante a rota ótima**: nenhuma possibilidade fica de fora. Dividir para Conquistar não foi adotada porque juntar as melhores rotas de partes do mapa não garante a melhor rota do mapa inteiro, o que exigiria verificações extras.

2.2 Passo a passo

1 Representar o mapa como um grafo: para cada cidade, guardar as cidades vizinhas e o custo da estrada até cada uma.

2 Escolher a cidade inicial e separá-la das demais.

3 Gerar, uma a uma, todas as permutações (ordens possíveis) das demais cidades.

4 Para cada permutação, montar a rota: cidade inicial, cidades na ordem da permutação e retorno à cidade inicial.

5 Percorrer a rota somando o custo de cada trecho. Se algum trecho não tiver estrada, descartar a rota.

6 Se a rota for válida e mais barata que a melhor encontrada até agora, guardá-la como a melhor.

7 Depois de testar todas as permutações, a melhor rota guardada é a resposta. Se nenhuma rota for válida, o mapa não tem solução.

2.3 Pseudocódigo

```
ALGORITMO CaixeiroViajanteForcaBruta
ENTRADA: grafo G (cidades e custo de cada estrada), cidade inicial s
SAÍDA:   melhor rota e o seu custo

 1  melhor_custo ← ∞
 2  melhor_rota  ← NULO
 3  outras ← cidades de G, exceto s            { n – 1 cidades }
 4  PARA CADA permutação P de outras FAÇA      { (n – 1)! vezes }
 5      rota   ← s, P[1], P[2], ..., P[n – 1], s
 6      custo  ← 0
 7      valida ← VERDADEIRO
 8      PARA i ← 1 ATÉ n FAÇA
 9          SE não existe estrada entre rota[i] e rota[i + 1] ENTÃO
10              valida ← FALSO
11              INTERROMPA
12          custo ← custo + peso(rota[i], rota[i + 1])
13      SE valida E custo < melhor_custo ENTÃO
14          melhor_custo ← custo
15          melhor_rota  ← rota
16  RETORNE melhor_rota, melhor_custo
```

2.4 Complexidade

O algoritmo testa $(n - 1)!$ permutações e, para cada uma, soma até n trechos. O tempo é, portanto, $O(n!)$, que cresce mais rápido que qualquer exponencial. A memória usada pela busca é $O(n)$, pois só a rota atual e a melhor rota ficam guardadas. A Tabela 2 usa a velocidade medida no Exemplo 2 (cerca de 1.390.000 permutações por segundo) para estimar o tempo em mapas maiores.

Tabela 2 – Crescimento do número de permutações com o número de cidades

Cidades (n)	Permutações testadas $(n - 1)!$	Tempo estimado
4	6	menos de 1 s
8	5.040	menos de 1 s
10	362.880	0,26 s (medido)
12	39.916.800	≈ 29 s
15	87.178.291.200	≈ 17 horas
20	$1{,}2 \times 10^{17}$	≈ 2.800 anos

Fonte: elaborado pelo autor. Tempos estimados a partir da execução do Exemplo 2.

Com 10 cidades a busca termina em menos de um segundo. Em torno de 15 cidades já leva horas, e com 20 cidades se torna inviável. Esse é o principal limite da Força Bruta.

2.5 Implementação em Python

A solução foi implementada em Python 3 no arquivo `caixeiro_viajante.py`. O grafo é um dicionário em que cada cidade aponta para as vizinhas e o custo de cada estrada. As permutações são geradas por `itertools.permutations`, da biblioteca padrão. As funções principais estão abaixo.

```
def montar_grafo(arestas):
    grafo = {}
    for origem, destino, custo in arestas:
        grafo.setdefault(origem, {})[destino] = custo
        grafo.setdefault(destino, {})[origem] = custo
    return grafo

def custo_da_rota(grafo, rota):
    total = 0
    for atual, proxima in zip(rota, rota[1:]):
        if proxima not in grafo[atual]:
            return None
        total += grafo[atual][proxima]
    return total

def caixeiro_forca_bruta(grafo, cidade_inicial):
    outras = sorted(cidade for cidade in grafo if cidade != cidade_inicial)
    melhor_rota = None
    melhor_custo = float("inf")
    testadas = 0
    validas = []  # lista de (rota, custo) das permutações que formam um ciclo existente

    for permutacao in itertools.permutations(outras):
        testadas += 1
        rota = (cidade_inicial,) + permutacao + (cidade_inicial,)
        custo = custo_da_rota(grafo, rota)
        if custo is None:
            continue
        validas.append((rota, custo))
        if custo < melhor_custo:
            melhor_custo = custo
            melhor_rota = rota

    return melhor_rota, melhor_custo, testadas, validas
```

3. Exemplos de entrada, execução e resposta

3.1 Exemplo 1 – quatro cidades (todas ligadas entre si)

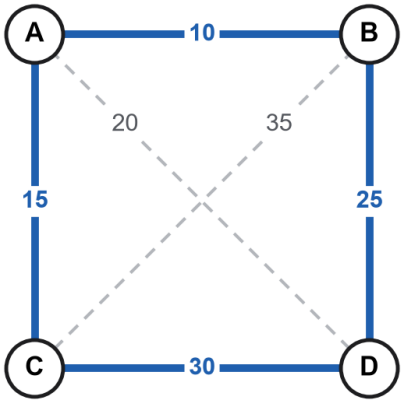
Um exemplo pequeno permite acompanhar todas as permutações à mão. **Entrada:** 4 cidades (A, B, C e D), todas ligadas entre si pelas 6 estradas da Tabela 3, com a cidade A como inicial.

Tabela 3 – Entrada do Exemplo 1

Estrada	A – B	A – C	A – D	B – C	B – D	C – D
Custo	10	15	20	35	25	30

Fonte: elaborado pelo autor.

Figura 2 – Exemplo 1 com a rota ótima destacada em azul



Fonte: elaborado pelo autor.

Execução. Com A fixa, o algoritmo testa as 3! = 6 permutações de B, C e D. A Tabela 4 mostra cada rota e o cálculo do seu custo.

Tabela 4 – Execução do Exemplo 1: todas as permutações testadas

#	Rota	Cálculo	Custo
1	A → B → C → D → A	10 + 35 + 30 + 20	95
2	A → B → D → C → A	10 + 25 + 30 + 15	**80**
3	A → C → B → D → A	15 + 35 + 25 + 20	95
4	A → C → D → B → A	15 + 30 + 25 + 10	**80**
5	A → D → B → C → A	20 + 25 + 35 + 15	95
6	A → D → C → B → A	20 + 30 + 35 + 10	95

Fonte: elaborado pelo autor.

Resposta. A rota de menor custo é **A → B → D → C → A**, com custo **80**. A rota A → C → D → B → A tem o mesmo custo porque é o mesmo ciclo percorrido no sentido contrário.

3.2 Exemplo 2 – grafo da atividade (10 cidades)

Entrada. O grafo da Figura 1, descrito na Tabela 1, com a cidade 1 como inicial. No programa, a entrada é a lista de estradas `ARESTAS_MAPA`, em que cada item é (cidade, cidade, custo).

Execução. O algoritmo testou as 9! = **362.880** permutações em 0,26 s. Dessas, 362.872 foram descartadas por usar um trecho sem estrada e apenas **8** formam uma rota válida (Tabela 5). As 8 rotas válidas correspondem a 4 ciclos diferentes, cada um percorrido nos dois sentidos.

Tabela 5 – Execução do Exemplo 2: rotas válidas entre as 362.880 testadas

#	Rota válida encontrada	Custo
1	1 → 2 → 4 → 5 → 3 → 6 → 7 → 9 → 10 → 8 → 1	**138**
2	1 → 3 → 2 → 4 → 5 → 6 → 7 → 9 → 10 → 8 → 1	139
3	1 → 3 → 5 → 4 → 2 → 10 → 8 → 9 → 7 → 6 → 1	143
4	1 → 3 → 5 → 4 → 2 → 10 → 9 → 7 → 6 → 8 → 1	149
5	1 → 6 → 7 → 9 → 8 → 10 → 2 → 4 → 5 → 3 → 1	143
6	1 → 8 → 6 → 7 → 9 → 10 → 2 → 4 → 5 → 3 → 1	149
7	1 → 8 → 10 → 9 → 7 → 6 → 3 → 5 → 4 → 2 → 1	**138**
8	1 → 8 → 10 → 9 → 7 → 6 → 5 → 4 → 2 → 3 → 1	139

Fonte: elaborado pelo autor, a partir da saída do programa.

Verificação. As cidades 4 e 7 têm apenas duas estradas cada (grau 2 na Tabela 1). Toda rota válida precisa entrar e sair delas, então as estradas 2–4, 4–5, 9–7 e 7–6 são obrigatórias. As oito rotas da Tabela 5 usam essas quatro estradas, o que confirma o resultado do programa.

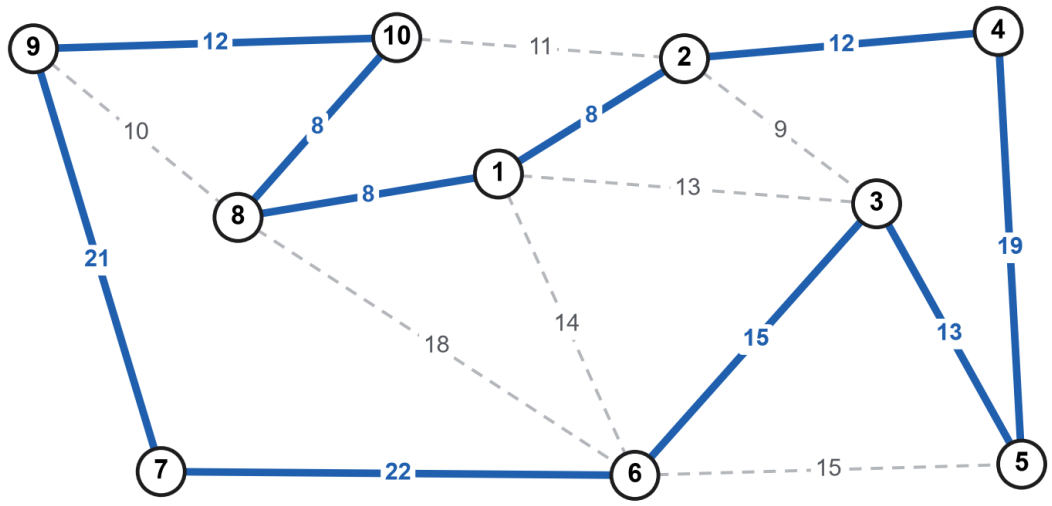
Resposta. A rota de menor custo é **1 → 2 → 4 → 5 → 3 → 6 → 7 → 9 → 10 → 8 → 1**, com custo total **138** (Tabela 6 e Figura 3). A rota 7 da Tabela 5 é o mesmo ciclo no sentido inverso.

Tabela 6 – Custo da rota ótima, trecho a trecho

Trecho	Custo	Acumulado
1 → 2	8	8
2 → 4	12	20
4 → 5	19	39
5 → 3	13	52
3 → 6	15	67
6 → 7	22	89
7 → 9	21	110
9 → 10	12	122
10 → 8	8	130
8 → 1	8	138
Total		**138**

Fonte: elaborado pelo autor.

Figura 3 – Rota ótima (custo 138) destacada em azul; estradas não usadas tracejadas



Fonte: elaborado pelo autor.

3.3 Saída do programa

Saída completa ao executar `python caixeiro_viajante.py`:

```
===== EXEMPLO 1 - 4 CIDADES =====
Cidades: 4 | Estradas: 6 | Cidade inicial: A
Permutações testadas: 6
Rotas válidas (todas as estradas existem): 6
  A -> B -> C -> D -> A = 95
  A -> B -> D -> C -> A = 80  <- menor custo
  A -> C -> B -> D -> A = 95
  A -> C -> D -> B -> A = 80  <- menor custo
  A -> D -> B -> C -> A = 95
  A -> D -> C -> B -> A = 95
Melhor rota: A -> B -> D -> C -> A
Custo total: 80
Tempo de execução: 0.000 s

===== EXEMPLO 2 - GRAFO DA ATIVIDADE =====
Cidades: 10 | Estradas: 17 | Cidade inicial: 1
Permutações testadas: 362880
Rotas válidas (todas as estradas existem): 8
  1 -> 2 -> 4 -> 5 -> 3 -> 6 -> 7 -> 9 -> 10 -> 8 -> 1 = 138  <- menor custo
  1 -> 3 -> 2 -> 4 -> 5 -> 6 -> 7 -> 9 -> 10 -> 8 -> 1 = 139
  1 -> 3 -> 5 -> 4 -> 2 -> 10 -> 8 -> 9 -> 7 -> 6 -> 1 = 143
  1 -> 3 -> 5 -> 4 -> 2 -> 10 -> 9 -> 7 -> 6 -> 8 -> 1 = 149
  1 -> 6 -> 7 -> 9 -> 8 -> 10 -> 2 -> 4 -> 5 -> 3 -> 1 = 143
  1 -> 8 -> 6 -> 7 -> 9 -> 10 -> 2 -> 4 -> 5 -> 3 -> 1 = 149
  1 -> 8 -> 10 -> 9 -> 7 -> 6 -> 3 -> 5 -> 4 -> 2 -> 1 = 138  <- menor custo
  1 -> 8 -> 10 -> 9 -> 7 -> 6 -> 5 -> 4 -> 2 -> 3 -> 1 = 139
Melhor rota: 1 -> 2 -> 4 -> 5 -> 3 -> 6 -> 7 -> 9 -> 10 -> 8 -> 1
Custo total: 138
Tempo de execução: 0.261 s
```

4. Considerações finais

A Força Bruta encontrou a rota ótima do grafo da atividade, **1 → 2 → 4 → 5 → 3 → 6 → 7 → 9 → 10 → 8 → 1**, com custo **138**. Como todas as 362.880 permutações foram testadas, não existe rota mais barata. A técnica é simples de implementar e garante a resposta exata, mas o número de permutações cresce de forma fatorial. Ela é adequada para mapas de até cerca de 12 cidades. Para mapas maiores, existem alternativas como a programação dinâmica de Held e Karp, que tem complexidade $O(n^2 \cdot 2^n)$ e ainda garante a rota ótima, e heurísticas como a do vizinho mais próximo, que são rápidas mas não garantem a melhor rota.

Referências

APPLEGATE, D. L.; BIXBY, R. E.; CHVÁTAL, V.; COOK, W. J. **The traveling salesman problem**: a computational study. Princeton: Princeton University Press, 2006.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Algoritmos**: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

HELD, M.; KARP, R. M. A dynamic programming approach to sequencing problems. **Journal of the Society for Industrial and Applied Mathematics**, v. 10, n. 1, p. 196-210, 1962.

LEVITIN, A. **Introduction to the design and analysis of algorithms**. 3. ed. Boston: Pearson, 2012.

PYTHON SOFTWARE FOUNDATION. **itertools**: functions creating iterators for efficient looping. Disponível em: https://docs.python.org/3/library/itertools.html. Acesso em: 22 out. 2026.

ZIVIANI, N. **Projeto de algoritmos**: com implementações em Pascal e C. 3. ed. São Paulo: Cengage Learning, 2011.